



National Technical University of Athens

School of Electrical and Computer Engineering

8th Semester

VLSI Digital Systems

4th Laboratory Exercise

Dimitris Vatalas

(Student ID: 03121093)

Aristotelis Georgios Dritsas

(Student ID: 03122127)

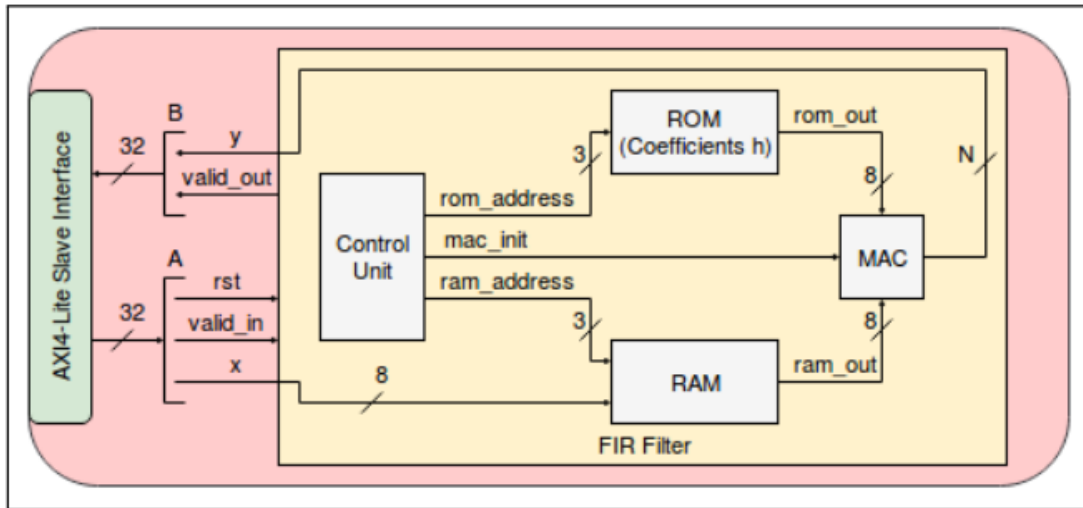
Athens, March 2026

Contents

Introduction	2
AXI4-Lite IP	2
AXI4-Lite Interface Code	2
Control Unit Code	10
MAC Unit Code	12
System Implementation and Bitstream Generation	13
Programming the ZYNQ SoC FPGA and Running the Application	14

Introduction

In this laboratory exercise, the code developed in the previous exercise, which implements an FIR filter, was adapted with the goal of deploying it on the ZYBO development board and operating it on real hardware. At the same time, appropriate code was developed for the embedded ARM processor, allowing the input data to be sent from the processor to the FPGA for processing, while the results are returned following the reverse path. Finally, communication between the ARM processor and the FPGA is implemented through an AXI4-Lite interface. The FIR filter has the following structure:



AXI4-Lite IP

Initially, we created the block design, integrating the ZYNQ Processing System, a custom IP with an AXI4-Lite interface in which the FIR filter is implemented, as well as the ILA, which allows the AXI bus to be monitored in case of potential problems. The code for the custom IP with the AXI4-Lite interface is presented below:

AXI4-Lite Interface Code

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity myip_final_v1_0_S00_AXI is
  generic (
    -- Users to add parameters here

    -- User parameters ends
    -- Do not modify the parameters beyond this line

    -- Width of S_AXI data bus
    C_S_AXI_DATA_WIDTH      : integer      := 32;
    -- Width of S_AXI address bus
    C_S_AXI_ADDR_WIDTH     : integer      := 4
  );
  port (
    -- Users to add ports here
```

```

-- User ports ends
-- Do not modify the ports beyond this line

-- Global Clock Signal
S_AXI_ACLK      : in std_logic;
-- Global Reset Signal. This Signal is Active LOW
S_AXI_ARESETN   : in std_logic;
-- Write address (issued by master, accepted by Slave)
S_AXI_AWADDR    : in std_logic_vector(C_S_AXI_ADDR_WIDTH-1 downto 0);
-- Write channel Protection type. This signal indicates the
--   privilege and security level of the transaction, and whether
--   the transaction is a data access or an instruction access.
S_AXI_AWPROT    : in std_logic_vector(2 downto 0);
-- Write address valid. This signal indicates that the master is
--   signaling a valid write address and control information.
S_AXI_AWVALID   : in std_logic;
-- Write address ready. This signal indicates that the slave is ready
--   to accept an address and associated control signals.
S_AXI_AWREADY   : out std_logic;
-- Write data (issued by master, accepted by Slave)
S_AXI_WDATA     : in std_logic_vector(C_S_AXI_DATA_WIDTH-1 downto 0);
-- Write strobes. This signal indicates which byte lanes hold
--   valid data. There is one write strobe bit for each eight
--   bits of the write data bus.
S_AXI_WSTRB     : in std_logic_vector((C_S_AXI_DATA_WIDTH/8)-1 downto 0);
-- Write valid. This signal indicates that valid write
--   data and strobes are available.
S_AXI_WVALID    : in std_logic;
-- Write ready. This signal indicates that the slave
--   can accept the write data.
S_AXI_WREADY    : out std_logic;
-- Write response. This signal indicates the status
--   of the write transaction.
S_AXI_BRESP     : out std_logic_vector(1 downto 0);
-- Write response valid. This signal indicates that the channel
--   is signaling a valid write response.
S_AXI_BVALID    : out std_logic;
-- Response ready. This signal indicates that the master
--   can accept a write response.
S_AXI_BREADY    : in std_logic;
-- Read address (issued by master, accepted by Slave)
S_AXI_ARADDR    : in std_logic_vector(C_S_AXI_ADDR_WIDTH-1 downto 0);
-- Protection type. This signal indicates the privilege
--   and security level of the transaction, and whether the
--   transaction is a data access or an instruction access.
S_AXI_ARPROT    : in std_logic_vector(2 downto 0);
-- Read address valid. This signal indicates that the channel
--   is signaling a valid read address and control information.
S_AXI_ARVALID   : in std_logic;
-- Read address ready. This signal indicates that the slave is
--   ready to accept an address and associated control signals.
S_AXI_ARREADY   : out std_logic;
-- Read data (issued by slave)
S_AXI_RDATA     : out std_logic_vector(C_S_AXI_DATA_WIDTH-1 downto 0);
-- Read response. This signal indicates the status of the
--   read transfer.
S_AXI_RRESP     : out std_logic_vector(1 downto 0);
-- Read valid. This signal indicates that the channel is
--   signaling the required read data.

```

```

        S_AXI_RVALID      : out std_logic;
        -- Read ready. This signal indicates that the master can
        -- accept the read data and response information.
        S_AXI_RREADY      : in std_logic
    );
end myip_final_v1_0_S00_AXI;

architecture arch_imp of myip_final_v1_0_S00_AXI is

    -- AXI4-Lite signals
    signal axi_awaddr      : std_logic_vector(C_S_AXI_ADDR_WIDTH-1 downto 0);
    signal axi_awready     : std_logic;
    signal axi_wready     : std_logic;
    signal axi_bresp       : std_logic_vector(1 downto 0);
    signal axi_bvalid     : std_logic;
    signal axi_araddr      : std_logic_vector(C_S_AXI_ADDR_WIDTH-1 downto 0);
    signal axi_arready     : std_logic;
    signal axi_rdata       : std_logic_vector(C_S_AXI_DATA_WIDTH-1 downto 0);
    signal axi_rresp       : std_logic_vector(1 downto 0);
    signal axi_rvalid     : std_logic;

    -- Example-specific design signals
    -- local parameter for addressing 32 bit / 64 bit C_S_AXI_DATA_WIDTH
    -- ADDR_LSB is used for addressing 32/64 bit registers/memories
    -- ADDR_LSB = 2 for 32 bits (n downto 2)
    -- ADDR_LSB = 3 for 64 bits (n downto 3)
    constant ADDR_LSB      : integer := (C_S_AXI_DATA_WIDTH/32)+ 1;
    constant OPT_MEM_ADDR_BITS : integer := 1;

    -----
    ---- Signals for user logic register space example
    -----

    ---- Number of Slave Registers 4
    signal slv_reg0         : std_logic_vector(C_S_AXI_DATA_WIDTH-1 downto 0);
    signal slv_reg1         : std_logic_vector(C_S_AXI_DATA_WIDTH-1 downto 0);
    signal slv_reg2         : std_logic_vector(C_S_AXI_DATA_WIDTH-1 downto 0);
    signal slv_reg3         : std_logic_vector(C_S_AXI_DATA_WIDTH-1 downto 0);
    signal slv_reg_rden     : std_logic;
    signal slv_reg_wren     : std_logic;
    signal reg_data_out     : std_logic_vector(C_S_AXI_DATA_WIDTH-1 downto 0);
    signal byte_index       : integer;
    signal aw_en           : std_logic;

    signal valid : std_logic;
    signal outp  : std_logic_vector(18 downto 0);

    component fir is
        Port (
            clk: in std_logic;
            rst: in std_logic;
            valid_in: in std_logic;
            x: in std_logic_vector (7 downto 0);
            valid_out: out std_logic;
            y: out std_logic_vector (18 downto 0)
        );
    end component;

begin

    FIR_axi: fir

```

```

port map (
    clk      => S_AXI_ACLK,
    rst      => slv_reg0(9),
    valid_in => slv_reg0(8),
    x        => slv_reg0(7 downto 0),
    valid_out => valid,
    y        => outp
);

-- outp <= slv_reg1(18 downto 0);
-- valid <= slv_reg1(19);
-- slv_reg1(31 downto 20) <= (others => '0');

-- I/O Connections assignments
S_AXI_AWREADY <= axi_awready;
S_AXI_WREADY <= axi_wready;
S_AXI_BRESP <= axi_bresp;
S_AXI_BVALID <= axi_bvalid;
S_AXI_ARREADY <= axi_arready;
S_AXI_RDATA <= axi_rdata;
S_AXI_RRESP <= axi_rresp;
S_AXI_RVALID <= axi_rvalid;
-- Implement axi_awready generation
-- axi_awready is asserted for one S_AXI_ACLK clock cycle when both
-- S_AXI_AWVALID and S_AXI_WVALID are asserted. axi_awready is
-- de-asserted when reset is low.

process (S_AXI_ACLK)
begin
    if rising_edge(S_AXI_ACLK) then
        if S_AXI_ARESETN = '0' then
            axi_awready <= '0';
            aw_en <= '1';
        else
            if (axi_awready = '0' and S_AXI_AWVALID = '1' and S_AXI_WVALID = '1' and aw_en = '1')
            then
                -- slave is ready to accept write address when
                -- there is a valid write address and write data
                -- on the write address and data bus. This design
                -- expects no outstanding transactions.
                axi_awready <= '1';
                aw_en <= '0';
            elsif (S_AXI_BREADY = '1' and axi_bvalid = '1') then
                aw_en <= '1';
                axi_awready <= '0';
            else
                axi_awready <= '0';
            end if;
        end if;
    end if;
end process;

-- Implement axi_awaddr latching
-- This process is used to latch the address when both
-- S_AXI_AWVALID and S_AXI_WVALID are valid.

process (S_AXI_ACLK)
begin
    if rising_edge(S_AXI_ACLK) then

```

```

    if S_AXI_ARESETN = '0' then
        axi_awaddr <= (others => '0');
    else
        if (axi_awready = '0' and S_AXI_AWVALID = '1' and S_AXI_WVALID = '1' and aw_en = '1')
            then
                -- Write address latching
                axi_awaddr <= S_AXI_AWADDR;
            end if;
        end if;
    end if;
end process;

-- Implement axi_wready generation
-- axi_wready is asserted for one S_AXI_ACLK clock cycle when both
-- S_AXI_AWVALID and S_AXI_WVALID are asserted. axi_wready is
-- de-asserted when reset is low.

process (S_AXI_ACLK)
begin
    if rising_edge(S_AXI_ACLK) then
        if S_AXI_ARESETN = '0' then
            axi_wready <= '0';
        else
            if (axi_wready = '0' and S_AXI_WVALID = '1' and S_AXI_AWVALID = '1' and aw_en = '1')
                then
                    -- slave is ready to accept write data when
                    -- there is a valid write address and write data
                    -- on the write address and data bus. This design
                    -- expects no outstanding transactions.
                    axi_wready <= '1';
                else
                    axi_wready <= '0';
                end if;
            end if;
        end if;
    end process;

-- Implement memory mapped register select and write logic generation
-- The write data is accepted and written to memory mapped registers when
-- axi_wready, S_AXI_WVALID, axi_wready and S_AXI_WVALID are asserted.
-- Write strobes are used to select byte enables of slave registers while writing.
-- These registers are cleared when reset (active low) is applied.
-- Slave register write enable is asserted when valid address and data are available
-- and the slave is ready to accept the write address and write data.
slv_reg_wren <= axi_wready and S_AXI_WVALID and axi_wready and S_AXI_AWVALID;

process (S_AXI_ACLK)
variable loc_addr :std_logic_vector(OPT_MEM_ADDR_BITS downto 0);
begin
    if rising_edge(S_AXI_ACLK) then
        if S_AXI_ARESETN = '0' then
            slv_reg0 <= (others => '0');
            slv_reg2 <= (others => '0');
            slv_reg3 <= (others => '0');
        else
            loc_addr := axi_awaddr(ADDR_LSB + OPT_MEM_ADDR_BITS downto ADDR_LSB);
            if (slv_reg_wren = '1') then
                case loc_addr is
                    when b"00" =>
                        for byte_index in 0 to (C_S_AXI_DATA_WIDTH/8-1) loop

```

```

        if ( S_AXI_WSTRB(byte_index) = '1' ) then
            -- Respective byte enables are asserted as per write strobes
            -- slave register 0
            slv_reg0(byte_index*8+7 downto byte_index*8) <= S_AXI_WDATA(byte_index*8+7
            ↪ downto byte_index*8);
        end if;
    end loop;
    when b"01" =>
        null;
    when b"10" =>
        for byte_index in 0 to (C_S_AXI_DATA_WIDTH/8-1) loop
            if ( S_AXI_WSTRB(byte_index) = '1' ) then
                -- Respective byte enables are asserted as per write strobes
                -- slave register 2
                slv_reg2(byte_index*8+7 downto byte_index*8) <= S_AXI_WDATA(byte_index*8+7
                ↪ downto byte_index*8);
            end if;
        end loop;
    when b"11" =>
        for byte_index in 0 to (C_S_AXI_DATA_WIDTH/8-1) loop
            if ( S_AXI_WSTRB(byte_index) = '1' ) then
                -- Respective byte enables are asserted as per write strobes
                -- slave register 3
                slv_reg3(byte_index*8+7 downto byte_index*8) <= S_AXI_WDATA(byte_index*8+7
                ↪ downto byte_index*8);
            end if;
        end loop;
    when others =>
        slv_reg0 <= slv_reg0;
        slv_reg2 <= slv_reg2;
        slv_reg3 <= slv_reg3;
    end case;
else
    slv_reg0(8) <= '0';
end if;
end if;
end if;
end process;

```

```

-- Implement write response logic generation
-- The write response and response valid signals are asserted by the slave
-- when axi_wready, S_AXI_WVALID, axi_wready and S_AXI_WVALID are asserted.
-- This marks the acceptance of the address and indicates the status of
-- the write transaction.

```

```

process (S_AXI_ACLK)
begin
    if rising_edge(S_AXI_ACLK) then
        if S_AXI_ARESETN = '0' then
            axi_bvalid <= '0';
            axi_bresp <= "00";
        else
            if (axi_awready = '1' and S_AXI_AWVALID = '1' and axi_wready = '1' and S_AXI_WVALID =
            ↪ '1' and axi_bvalid = '0') then
                axi_bvalid <= '1';
                axi_bresp <= "00";
            elsif (S_AXI_BREADY = '1' and axi_bvalid = '1') then
                axi_bvalid <= '0';
            end if;
        end if;
    end if;
end process;

```

```

    end if;
end process;

-- Implement axi_arready generation
-- axi_arready is asserted for one S_AXI_ACLK clock cycle when
-- S_AXI_ARVALID is asserted. axi_arready is de-asserted when
-- reset (active low) is asserted.
-- The read address is also latched when S_AXI_ARVALID is asserted.

process (S_AXI_ACLK)
begin
    if rising_edge(S_AXI_ACLK) then
        if S_AXI_ARESETN = '0' then
            axi_arready <= '0';
            axi_araddr <= (others => '1');
        else
            if (axi_arready = '0' and S_AXI_ARVALID = '1') then
                -- Indicates that the slave has accepted the valid read address
                axi_arready <= '1';
                -- Read address latching
                axi_araddr <= S_AXI_ARADDR;
            else
                axi_arready <= '0';
            end if;
        end if;
    end if;
end process;

-- Implement axi_rvalid generation
-- axi_rvalid is asserted for one S_AXI_ACLK clock cycle when both
-- S_AXI_ARVALID and axi_arready are asserted. The slave register
-- data are available on the axi_rdata bus at this instance.
-- The assertion of axi_rvalid marks the validity of read data on the
-- bus and axi_rresp indicates the status of the read transaction.
-- axi_rvalid is deasserted on reset (active low).

process (S_AXI_ACLK)
begin
    if rising_edge(S_AXI_ACLK) then
        if S_AXI_ARESETN = '0' then
            axi_rvalid <= '0';
            axi_rresp <= "00";
        else
            if (axi_arready = '1' and S_AXI_ARVALID = '1' and axi_rvalid = '0') then
                -- Valid read data is available at the read data bus
                axi_rvalid <= '1';
                axi_rresp <= "00"; -- 'OKAY' response
            elsif (axi_rvalid = '1' and S_AXI_RREADY = '1') then
                -- Read data is accepted by the master
                axi_rvalid <= '0';
            end if;
        end if;
    end if;
end process;

-- Implement memory mapped register select and read logic generation
-- Slave register read enable is asserted when a valid address is available
-- and the slave is ready to accept the read address.
slv_reg_rden <= axi_arready and S_AXI_ARVALID and (not axi_rvalid);

process (slv_reg0, slv_reg1, slv_reg2, slv_reg3, axi_araddr, S_AXI_ARESETN, slv_reg_rden)

```

```

variable loc_addr :std_logic_vector(OPT_MEM_ADDR_BITS downto 0);
begin
    -- Address decoding for reading registers
    loc_addr := axi_araddr(ADDR_LSB + OPT_MEM_ADDR_BITS downto ADDR_LSB);
    case loc_addr is
        when b"00" =>
            null;
        when b"01" =>
            reg_data_out <= slv_reg1;
        when b"10" =>
            reg_data_out <= slv_reg2;
        when b"11" =>
            reg_data_out <= slv_reg3;
        when others =>
            reg_data_out <= (others => '0');
    end case;
end process;

-- Output register or memory read data
process (S_AXI_ACLK) is
begin
    if (rising_edge(S_AXI_ACLK)) then
        if (S_AXI_ARESETN = '0') then
            axi_rdata <= (others => '0');
        else
            if (slv_reg_rden = '1') then
                -- When there is a valid read address (S_AXI_ARVALID) with
                -- acceptance of the read address by the slave (axi_arready),
                -- output the read data.
                axi_rdata <= reg_data_out;
            end if;
        end if;
    end if;
end process;

process (S_AXI_ACLK) is
begin
    if rising_edge(S_AXI_ACLK) then
        if S_AXI_ARESETN = '0' then
            slv_reg1 <= (others => '0');
        elsif valid = '1' then
            slv_reg1 <= (31 downto 20 => '0') & valid & outp;
        end if;
    end if;
end process;

end arch_imp;

```

In the AXI4-Lite interface code, the FIR filter component was initially integrated into the system, and the intended port mapping was implemented according to the specifications, as shown below:

- $A[7:0] = x$
- $A[8] = \text{valid_in}$
- $A[9] = \text{rst}$
- $A[31:10] = \text{not used}$
- $B[N-1:0] = y$
- $B[N] = \text{valid_out}$
- $B[31:N+1] = \text{not used}$

In our implementation, A corresponds to `slv_reg0`, while B corresponds to `slv_reg1`.

Additional modifications were then introduced to ensure the correct operation of the `valid_in` and `valid_out` signals. More specifically, in the AXI4-Lite interface code:

- writing to `slv_reg1`, which corresponds to the filter output, was disabled,
- the `valid_in` signal was appropriately handled so that it is automatically cleared after each new write operation performed by the ARM processor.

These modifications ensure the smooth and reliable operation of the filter.

Control Unit Code

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.std_logic_unsigned.all;

entity control_unit is
    Port(
        valid_in: in std_logic;
        clk: in std_logic;
        rst: in std_logic;

        en: out std_logic;
        we: out std_logic;
        mac_init: out std_logic;
        rom_addr: out std_logic_vector(2 downto 0);
        ram_addr: out std_logic_vector(2 downto 0);
        valid_out: out std_logic;
        mac_en: out std_logic
    );
end control_unit;

architecture Behavioral of control_unit is

    signal count: std_logic_vector(2 downto 0) := "000";
    signal enabled_flag: std_logic := '0';

begin
```

```

process(clk, rst, valid_in)
begin
    if rst = '1' then
        count <= "000";
        mac_init <= '1';
        valid_out <= '0';
        enabled_flag <= '0';
        en <= '0';

    elsif (enabled_flag = '0' and valid_in = '1') then
        enabled_flag <= '1';
        en <= '1';

    elsif (rising_edge(clk) and enabled_flag = '1') then

        if valid_in = '1' and count = "000" then
            count <= count + 1;
            mac_init <= '1';
            valid_out <= '1';
            mac_en<='1';

        elsif valid_in = '1' and count > "000" then
            mac_init <= '0';
            valid_out <= '0';
            count <= "000";
            mac_en<='1';

        elsif valid_in = '0' and count ="000" then
            mac_init <= '0';
            valid_out <= '1';
            count <= "000";
            mac_en<='0';
        else
            mac_init <= '0';
            valid_out <= '0';
            count <= count + 1;
            mac_en<='1';
        end if;

    end if;

end if;
end process;

we <= valid_in;
rom_addr <= count;
ram_addr <= count;

end Behavioral;

```

The Control Unit was modified in order to correctly handle input samples that are separated by more than 8 clock cycles, as is the case for signals originating from the ARM processor. At the same time, the operation of the valid_out signal was modified so that it remains asserted from the moment the FPGA produces a valid result until the reception of the next valid_in signal from the ARM processor. In this way, it is ensured that the ARM processor can reliably detect and read the valid_out signal.

MAC Unit Code

To support the modified control logic described above, an additional enable control signal, `mac_en`, was introduced in the MAC unit, as shown in the following code. This signal is controlled by the Control Unit and enables or disables the MAC operation.

More specifically, the MAC unit is disabled once the current calculation has been completed and remains in this state until a new `valid_in` signal arrives. This ensures that the correct result remains available at the output while `valid_out` remains asserted.

When a new `valid_in` signal arrives, it indicates the beginning of a new calculation. At the same time, it implies that the ARM processor has successfully read the previous result. Therefore, `valid_out` is deasserted by the Control Unit and the MAC unit is enabled again, starting the new calculation. In this way, proper synchronization between the ARM processor and the FPGA is achieved.

It is worth noting that the above procedure has been implemented in such a way that, when input data arrive every 8 clock cycles, the specifications of the previous laboratory exercise are fully preserved. More specifically, in this case, the FIR filter produces a new result every 8 clock cycles without any impact on its functionality.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.std_logic_unsigned.all;

entity mac is
    Port (
        a: in std_logic_vector (7 downto 0);
        b: in std_logic_vector (7 downto 0);
        clk: in std_logic;
        mac_init: in std_logic;

        y: out std_logic_vector (18 downto 0);
        mac_en : in std_logic
    );
end mac;

architecture Behavioral of mac is

    signal accumulator: std_logic_vector(18 downto 0) := (others => '0');

begin

    process(clk)
    begin
        if rising_edge(clk) then
            if mac_en = '0' then
                accumulator<=accumulator;
            else
                if mac_init = '1' then
                    accumulator <= "000" & (a * b);
                else
                    accumulator <= accumulator + (a * b);
                end if;
            end if;
        end if;
    end process;

end process;
```

```

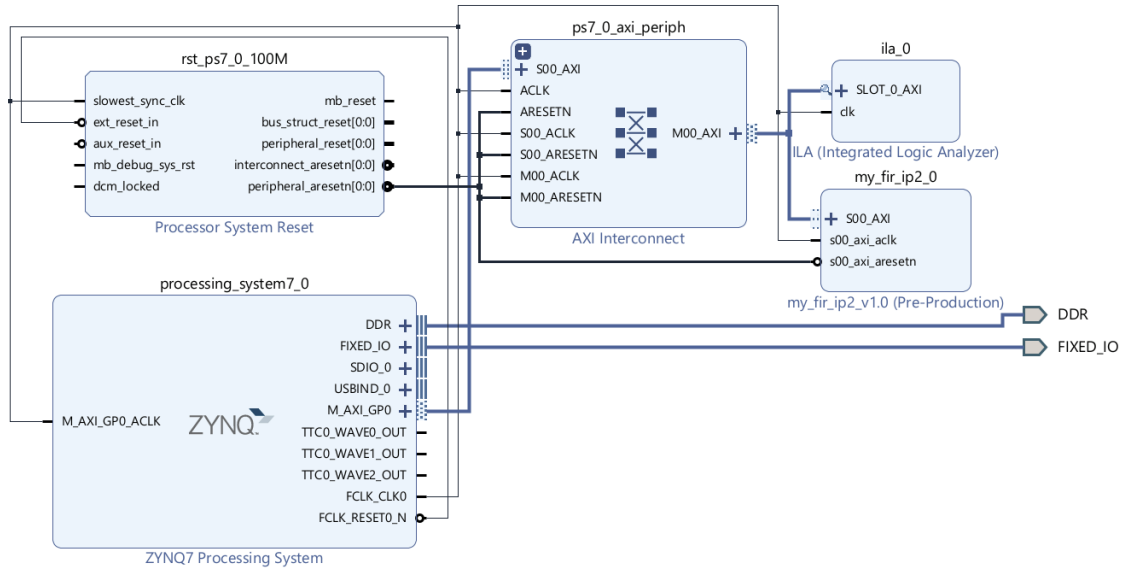
y <= accumulator;

end Behavioral;

```

System Implementation and Bitstream Generation

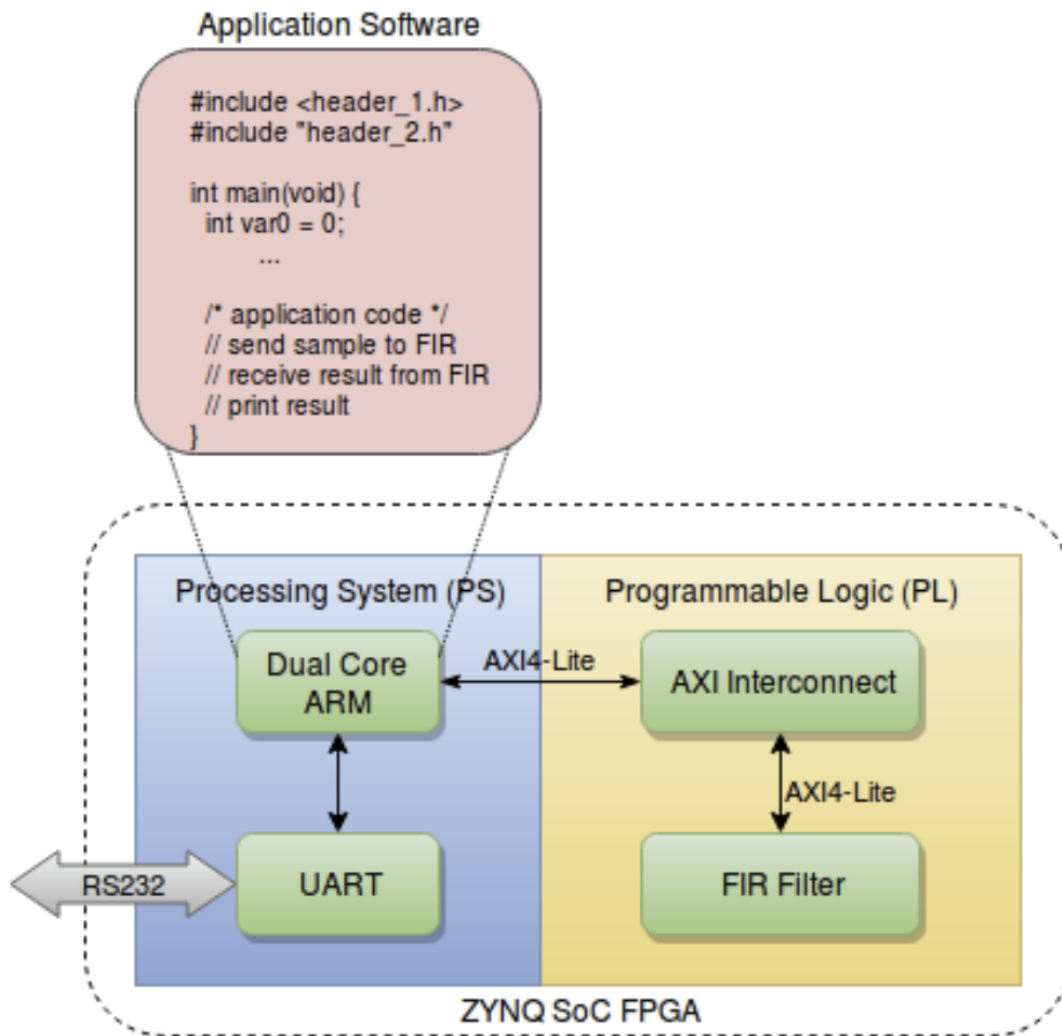
After completing the implementation of the filter, the modifications to the AXI interface, and the integration of the filter within the AXI interface, we generate the IP package. After making the necessary connections between the ZYNQ Processing System, the AXI4-Lite interface, and the ILA, the following block design is obtained:



The bitstream can now be generated using Vivado. Subsequently, the hardware can be exported using the “Include Bitstream” option. This process results in the creation of an **.xsa** file. The generated file can then be imported into Vitis, through which the complete ZYBO system can be programmed, including both the ARM processor and the FPGA.

Programming the ZYNQ SoC FPGA and Running the Application

The overall architecture of our system is presented below, followed by the code developed for the ARM processor to establish communication with the FPGA. More specifically, this code is responsible for transmitting the input data to the FPGA and subsequently receiving the corresponding results.



Εικόνα 2 : Αρχιτεκτονική συστήματος.

```

#include <stdio.h>
#include "platform.h"
#include "xil_printf.h"
#include "xparameters.h"
#include "xil_io.h"
#include "xil_types.h"
#include "sleep.h"

#define FIR_BASEADDR    XPAR_MYIP_FINAL_O_SOO_AXI_BASEADDR
#define FIR_IN_OFFSET   0x0
#define FIR_OUT_OFFSET  0x4

int main()
{
    init_platform();

    u8 inputs[] = {107, 90, 224, 153, 54, 132, 130, 78, 48, 145, 151, 222, 164, 168, 155, 209,
                    160, 243, 158, 102, 0, 0, 0, 0, 0, 0, 0, 0};

    u32 receive;
    u32 word;

    Xil_Out32(FIR_BASEADDR + FIR_IN_OFFSET, (1 << 9));
    usleep(100000);
    Xil_Out32(FIR_BASEADDR + FIR_IN_OFFSET, (0 << 9));

    printf("reset\r\n");
    for (int i = 0; i < 27; i++)
    {
        printf("iteration = %d\r\n", i);

        word = (1 << 8) | inputs[i];

        Xil_Out32(FIR_BASEADDR + FIR_IN_OFFSET, word);

        do {
            receive = Xil_In32(FIR_BASEADDR + FIR_OUT_OFFSET);
            usleep(20000);

        } while (((receive >> 19) & 0x1) == 0);

        u32 result = receive & 0x7FFFF;

        printf("in = %3u -> result = %u\r\n", inputs[i], result);
    }

    printf("Finished\r\n");

    cleanup_platform();
    return 0;
}

```

The input signals and the corresponding filter outputs are presented below:

$x = \{107, 90, 224, 153, 54, 132, 130, 78, 48, 145, 151, 222, 164, 168, 155, 209, 160, 243, 158, 102, 0, 0, 0, 0, 0, 0\}$

$y = \{107, 304, 625, 1199, 1860, 2731, 3720, 4786, 4468, 4103, 3949, 3886, 4515, 5138, 5934, 6715, 6906, 7037, 7275, 7046\}$

The FIR filter coefficients used in the implementation are:

$$h = \{1, 2, 3, 4, 5, 6, 7, 8\}$$

The results are shown below and confirm the correct implementation of the filter:

```
reset
iteration = 0
in = 107 -> result = 107
iteration = 1
in = 90 -> result = 304
iteration = 2
in = 224 -> result = 725
iteration = 3
in = 153 -> result = 1299
iteration = 4
in = 54 -> result = 1927
iteration = 5
in = 132 -> result = 2687
iteration = 6
in = 130 -> result = 3577
iteration = 7
in = 78 -> result = 4545
iteration = 8
in = 48 -> result = 4598
iteration = 9
in = 145 -> result = 4842
iteration = 10
in = 151 -> result = 3941
iteration = 11
in = 222 -> result = 3677
iteration = 12
in = 164 -> result = 4315
iteration = 13
in = 168 -> result = 4365
iteration = 14
in = 155 -> result = 4456
iteration = 15
in = 209 -> result = 5094
iteration = 16
in = 160 -> result = 6084
iteration = 17
in = 243 -> result = 6396
iteration = 18
in = 158 -> result = 6667
iteration = 19
in = 102 -> result = 6250
iteration = 20
```

```
in = 0 -> result = 6133
iteration = 21
in = 0 -> result = 5816
iteration = 22
in = 0 -> result = 5448
iteration = 23
in = 0 -> result = 4439
iteration = 24
in = 0 -> result = 3662
iteration = 25
in = 0 -> result = 1978
iteration = 26
in = 0 -> result = 816
iteration = 27
in = 0 -> result = 0
Finished
```